lecture → 2

Variable

- by `let` keyword
- by `const` keyword

let → Variable can be changed

const → Can not changed.

1-1 > JS 1.js > ...

```
1   // variable
2   //1- let->can change
3
4
5   let name="bhupesh";
6   let age=20;
7   console.log(name,age);
8
9   //2- const->cannot change
10  const account=2345.9;
11  console.log(account);
12  //account=3452.45;
13  //console.log(account);
14
15  // 3- var -> now it is not used
16  var a=10;
17  console.log(a);
18  var a=20;
19  console.log(a);
```

var →

Problems with var

(a) Can be declare variable n no. of times

(b) can run outside the scope

↓

(1) can work with global scope

(2) Can work with functional scope

```
if(true){
    var a=20;
}
console.log(a);
```

→ but scope of a should be block

→ will run

```
function fun(){
    var c=20;
}
console.log(c);
```

```
var b=30;
console.log(b);
```

```
// 3- var -> now it is not used
var a=10;
console.log(a);
var a=20;
console.log(a);

// problems with var

var j=34;
console.log(j);

/*function fun(){
    var z=23;
}
console.log(z); // z is not defined because it is block scope
*/
// but it fails here
if(true){
    var x=67;
}
console.log(x);
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[Done] exited with code=1 in 0.051 seconds

[Running] node "/Users/Bhupesh/Desktop/Shared/Files From c.localized/fr
bhupesh 20
2345.9
10
20
34
67

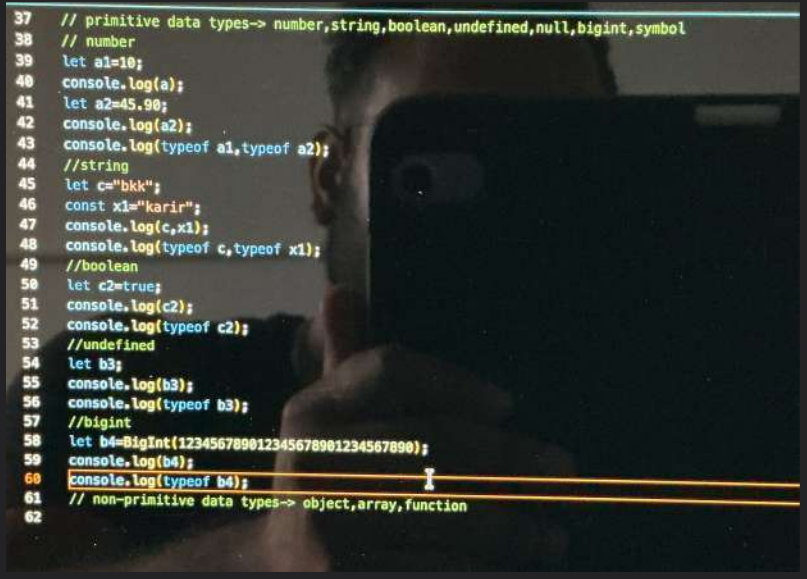
[Done] exited with code=0 in 0.039 seconds
```

⇒ A no. take 8 bytes of memory

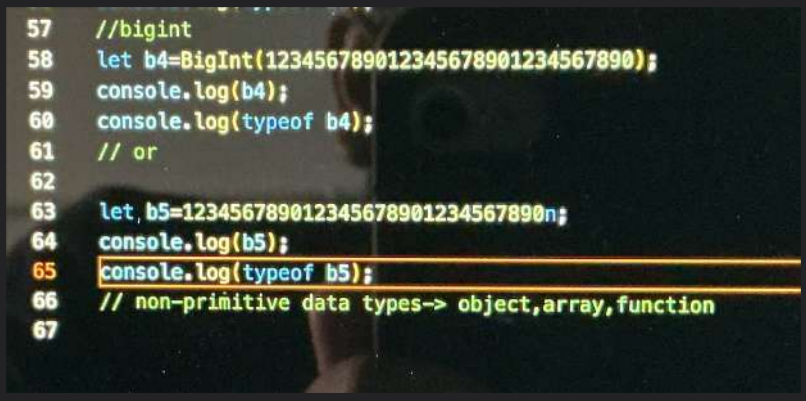
so if we unable to store number then use BigInt

BigInt

-2^{53} to $2^{53}-1$

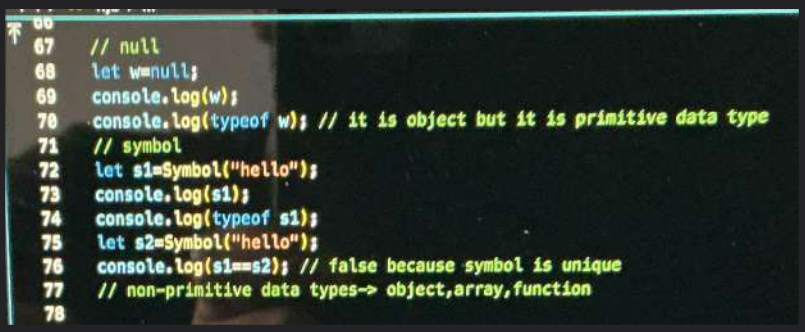


```
// primitive data types-> number,string,boolean,undefined,null,bigint,symbol
// number
let a1=10;
console.log(a);
let a2=45.90;
console.log(a2);
console.log(typeof a1,typeof a2);
//string
let c="bkk";
const x1="karir";
console.log(c,x1);
console.log(typeof c,typeof x1);
//boolean
let c2=true;
console.log(c2);
console.log(typeof c2);
//undefined
let b3;
console.log(b3);
console.log(typeof b3);
//bigint
let b4=BigInt(12345678901234567890123456789O);
console.log(b4);
console.log(typeof b4);
// non-primitive data types-> object,array,function
```



```
//bigint
let b4=BigInt(12345678901234567890123456789O);
console.log(b4);
console.log(typeof b4);
// or

let b5=12345678901234567890123456789On;
console.log(b5);
console.log(typeof b5);
// non-primitive data types-> object,array,function
```



```
// null
let w=null;
console.log(w);
console.log(typeof w); // it is object but it is primitive data type
// symbol
let s1=Symbol("hello");
console.log(s1);
console.log(typeof s1);
let s2=Symbol("hello");
console.log(s1==s2); // false because symbol is unique
// non-primitive data types-> object,array,function
```

Primitive data types =>

- array
- object
- function

① let arr = [1, 2, 3, "Bhupen"];
console.log(typeof arr); → object

② Object

// Rohit 1231 18 gen → what is meaning of it

so Object is present in the key value pairs i.e

③ function =>

```
function add(a,b){
    return a+b;
}
const cv = add(10,20);
console.log(cv);
```

Note => we can also store any function in variable

```
// non-primitive data types-> object,array,function
// array
let arr=[1,2,3,4,5,"bhupesh"];
console.log(arr);
console.log(typeof arr);

const arr1=[1,2,3,4,5];
arr1.push(6);
console.log(arr1);
// arr1=[1,2,3,4,5,6]; // error because arr1 is const
// object
let obj={
    name:"bhupesh",
    account_number:2345,
    age:25,
    category:"general"
}
console.log(obj);
// function
function add(a,b){
    return a+b;
}
const cv1=add(10,20);
console.log(cv1);
// or
let cv2=function sub(a,b){
    if(a>b) return a-b;
    else if(a==b) return 0;
    else return b-a;
}
const cv3=cv2(10,20);
console.log(cv3);
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[ 1, 2, 3, 4, 5, 'bhupesh' ]
object
[ 1, 2, 3, 4, 5, 6 ]
{ name: 'bhupesh', account_number: 2345, age: 25, category: 'general' }
30
10

[Done] exited with code=0 in 0.046 seconds
```

What is a famous bug in Javascript ?

=> ① let a = null;
console.log(typeof a); → object

=> (2) All typeof primitive data type is object only

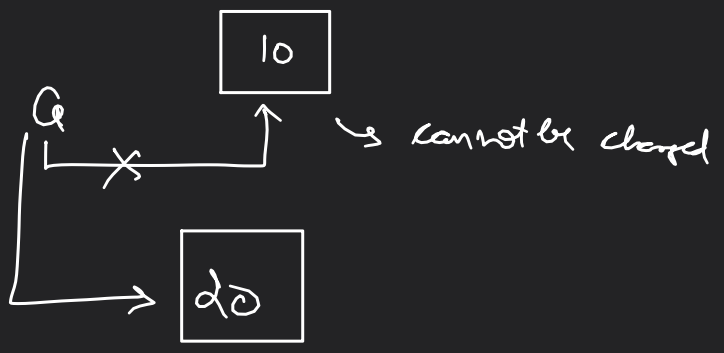
=> let arr = [1, 2, "Ram"];
console.log(typeof arr);
→ object ✓

=> let c = function(a, b){
return a-b;
}
console.log(typeof c);
→ function ?!

As the above statement says
every primitive data type is object
then why function is of type function
we will see in the coming lecture.

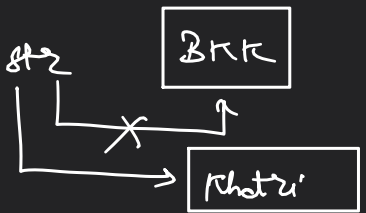
(3) Primitive data types (all 8) are
immutable data types

let a = 10;
a = 20; →



→ cannot be changed

let str = "Ram";
str = "Rhatri";



How to Proceed?

```
112
113  let str="bkk";
114  console.log(str[0]); // b
115  str[0]="k"; // string is immutable
116  console.log(str); // bkk
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
{ name: 'bhupesh', account_number: 2345, age: 25, category: 'gen
30
10
function
b
bkk

[Done] exited with code=0 in 0.055 seconds
```

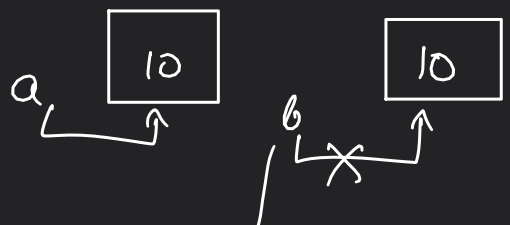
① Non Primitive data types are mutable

```
119  let arr12=[1,2,3,false,"bkk"];
120  arr12.push(4);
121  console.log(arr12); // [ 1, 2, 3, false, 'bkk', 4 ]
122  arr12[0]=10;
123  console.log(arr12); // [ 10, 2, 3, false, 'bkk', 4 ]
124
125  let obj12={
126      name:"bhupesh",
127      age:20
128  }
129  obj12.name="khatri";
130  console.log(obj12.name); // khatri
```

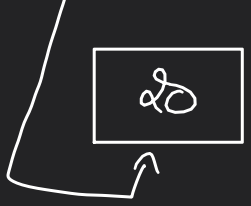
② Pass by value (Primitive)

let a = 10;

let b = a;



b=20;

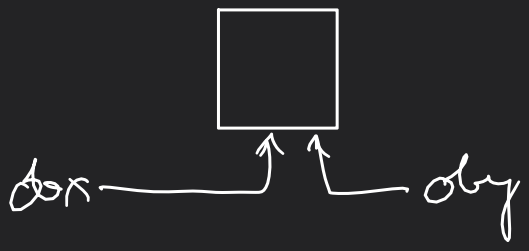



```
131 //-------------------------------
132 //copy by value-> in primitive data types
133 let a8=12;
134 let b8=a;
135 b8=20;
136 console.log(a8,b8); // 12 20
```

```
12 20

[Done] exited with code=0 in 0.054 seconds
```

⑥ Pass by refrence (Non-Primitive data types)




```
// pass by refrence->non primitive data type
let obx={
    name:"bkk",
    age:25
}
let oby=obx;
oby.name="khatri";
console.log(obx.name,oby.name); // khatri khatri
```

Lecture=>3

Memory Management

